A Profile-Guided Optimization to Reduce False Memory Dependence Predictions

MICRO 2026 Submission #**NaN** – Confidential Draft – Do NOT Distribute!!

Abstract

Memory Dependence Prediction (MDP) is a speculative technique to determine which stores, if any, a given load will depend on. Area-constrained cores are increasingly relevant in various applications such as energy-efficient or edge systems, and can often only spare limited space for MDP tables. This leads to a high number of false dependencies as memory independent loads collide with unrelated predictor entries (i.e., the aliasing problem), causing unnecessary stalls in the processor pipeline.

This paper proposes a profile-guided hardware-software co-design to detect frequently memory independent loads. Our profiler labels those loads via their opcode to mark them as *predict-no-dependency* (PND) loads. PND loads bypass MDP and are always scheduled as early as possible, eliminating the possibility of false dependencies without introducing additional hardware overhead. Across SPEC CPU 2017 intspeed, this technique achieves a geomean IPC gain of 1.4% in the smallest simulated core. On larger cores, performance is maintained, while average MDP power usage is reduced by 40%.

- do we need varying CPU sizes? maybe it'd make sense just to present the best result then have a size analysis subsection.
- have a section on finding the minimum store set size

1 Introduction

-turned into a false dependency/IPC story. is it still potentially worthwhile to try and push the MDP lookup reduction/predictor activity as a result? its only useful in so far as it reduces false deps (as percent power usage of MDP is so small), but maybe it could still be seen as interesting in its own right

Memory-level parallelism (MLP) is important for efficient out-of-order (OoO) execution and overcoming the memory wall in processor performance. Memory dependence prediction is a speculative technique used in OoO processors that aims to enhance total MLP by predicting the stores on which a given load instruction will depend, as well as identifying memory-independent loads. Smaller cores utilising smaller predictors can often struggle with a large number of false dependencies, which unnecessarily stalls loads on non-dependent stores. This is due to hash collisions in small predictor tables, or simple predictor algorithms that cannot capture varying dependency behaviour. The Store Sets predictor [**?**] is exhibits both of these issues, and variations of the algorithm are found in real processors today [**?**]. Whereas modern predictors like PHAST [**?**] have pushed the SotA to address these issues via techniques like associative tagging and context based predictions, they place higher area, energy and complexity demands on core design, which is often not viable in smaller cores. At the same time, while simply growing a Store Sets-like predictor would offset the hash collisions, this benefit struggles to outperform an iso-area comparison to scaling other components. As such, memory dependence predictors in small cores typically remain small and inefficient.

At the same time, the importance of such area-constrained cores has grown over time. Modern systems increasingly deploy heterogeneous processors containing both performance and efficiency-oriented core designs, combining larger high-performance cores with small energy-efficient (but still out-of-order) cores, as seen in ARM big.LITTLE systems and both Apple's & Intel's performance and efficiency cores. Efficiency-focused cores are, of course, also found in power-constrained systems such as smartphones, tablets, smartwatches and other edge systems. These cores are more and more performing demanding general purpose computation while striving to consume as little energy as possible. As they grow more performance-sensitive, finding optimisations which don't also incur additional area, power or complexity costs becomes more pressing.

This paper builds on an observation demonstrated by various prior works that a considerable portion of load instructions in general-purpose workloads rarely or never exhibit in-flight memory dependencies [**? ? ?**]. This is exploited to reduce false dependencies in memory dependence prediction via a profile-guided software co-design at **zero hardware cost**. Our technique labels frequently memory-independent loads by their opcode, causing them to bypass MDP queries and always schedule as early as possible. We show that the vast majority of dynamic loads can avoid querying the MDP while maintaining or improving IPC, as well as provide a potential power saving.

1.1 Contributions

This paper makes the following contributions:

- Implements a profile-guided co-design technique to label memory independent loads to the OoO core at no hardware overhead.
- Implements a modified Gem5 to simulate labelled loads against different MDP algorithms and core size configurations.
- Extends McPat to provide power usage estimations of the MDP unit.
- Evaluates Spec2017 IntSpeed and demonstrates on average:
  - IPC improves on the smallest core by 1.47%,
  - 71% of MDP load queries are redundant,
  - MDP power reduces by 12% on the smallest core, whereas the largest core maintains IPC and reduces MDP power by 42%.
- Additionally demonstrates a x% IPC gain when modelling a limited number of MDP read ports.

MICRO 2026, Athens, Greece
2026. ACM ISBN 978-X-XXXX-XXXX-X/XX/XX
https://doi.org/XXXXXXX.XXXXXXX

2 Background

This section outlines the fundamental concepts in memory-level parallelism for out-of-order execution. It then introduces Store Sets as a foundational memory dependence predictor algorithm, a more modern variation found in the XiangShan processor, and lastly the current state of the art in the PHAST predictor.

2.1 Loads in Out-of-Order Execution

The key structure in memory-level parallelism is the load-store queue (LSQ), used to hold in-flight memory operations and perform memory disamgibuation. When load instructions dispatch, they are inserted into the load queue (LQ), and query the memory dependence predictor (MDP) for a predicted store(s) to wait on before execution. Once their source operands are ready and all predicted dependent stores have executed, loads search backwards through the store queue (SQ) for store-forwarding opportunities. If no matching addresses are found, the loads access cache to get their data.

When a load searches the SQ, not all earlier stores may have resolved their addresses. As the majority of the time the address will not match with the load, it is often profitable to speculatively ignore these stores and issue the load as soon as possible. When the unresolved store eventually receives it's address, it searches forward through the LQ to find loads with matching addresses which have already executed. If a match is found, the load has read stale data and a memory order violation has occured. The processor must flush all in-flight instructions from the load onwards and re-fetch.

The goal of the MDP is to minimise violation events while maximising the available MLP, allowing memory independent loads to issue as soon as possible and dependent loads to wait only for the necessary stores. When a load is incorrectly stalled on a store that does not resolve to the same address this is called a false dependency. Simple MDP algorithms typically prioritise reducing the number of violations over the number of false dependencies.

2.2 Store Sets

A seminal paper in memory dependence prediction is 'Memory Dependence Prediction using Store Sets' [**?**]. This is the MDP algorithm implemented in the open source Gem5 simulator [**?**]. Store Sets works by using Store Set IDs (SSID) to track dependent load-store pairs, assigning each instruction the same ID value in the PC-indexed Store Set ID Table (SSIT). The SSIDs are then used to index the Last-Fetched Store Table (LFST), which holds the sequence number of the most recently issued store with an SSID mapping to that LFST entry. Newly issued loads then access the LFST via their SSID to find the store they're predicted to be dependent on. If a load PC does not map to a valid SSID entry, it is predicted to be memory independent. To forget old dependencies and reduce table pressure, the SSIT and LFST are cyclically cleared after a certain number of issued memory operations.

Store Sets is extremely area efficient, delivering a large portion of available performance with a very small hardware budget. It's small size and simplicity make it well suited to area-constrained processors. However, especially at smaller sizes, Store Sets struggles with false dependencies due hash collisions. When a memory independent load hashes to an existing SSID entry, it is incorrectly made dependent on the corresponding store for that entry. The clear period can be made shorter to offset this, but this comes at the trade-off of a higher number of memory order violations as the predictor must re-learn true dependencies more often. Furthermore, many loads exhibit non-consistent memory dependencies. A load in a loop may be dependent on a store every even iteration, but not on odd iterations. Store Sets has no way to disambiguate these cases, and conservatively predicts the load as dependent on every iteration.

2.3 XiangShan Store Sets

XiangShan is an open-source high-performance RISCV processor RTL implementation [**?**]. XiangShan represents the cutting edge in open-source superscalar processor design, and comes with a Gem5 fork modified to closer model the RTL implementation [**?**]. This Gem5 fork implements a variation of the Store Sets predictor that offers a closer representation to implementations found in real processors. From here on this is referred to as XS Store Sets.

The main optimisation over original Store Sets is using multiple 'slots' for each LFST entry, thereby recording multiple dependent stores at once. This allows a load to track multiple dependencies with only one SSID. This is useful in situations where a load is dependent on different stores in different program contexts, or may have partial address overlap with multiple stores. There are further small optimisations to allocation policy which we omit here.

2.4 PHAST

PHAST is a newer predictor which achieves much greater accuracy than Store Sets and other previously proposed MDP algorithms [**?**]. PHAST uses a TAGE-like structure to record a geometric series of branch history lengths between dependent load-store pairs. On dispatch, load instructions search all history length tables in parallel and selects the dependencies on the longest path (or predicts no dependency if no tables hit). This scheme addresses the previous problems in Store Sets by using multi-way tagged associative entries to reduce hash collisions, and predicting dependencies based on branch context to capture more elaborate dependency patterns. At a storage budget of 14.5KB, PHAST is shown to achieve within 1.5% accuracy of an ideal predictor.

Although PHAST delivers much greater accuracy, it also comes with greater hardware overhead. Though PHAST greatly outperforms Store Sets for iso-area comparisons at larger budgets, it struggles with the smaller budgets most efficiency-focused cores allow for memory dependence prediction. Furthermore, PHAST incurs pipeline complexity by requiring global branch history be made available in the load-store unit, has greater prediction latency, and consumes more power for the same area. This makes PHAST better suited only for larger performance-focused cores that would benefit most from the greater MLP.

3 Related Work

Improving Memory Dependence Prediction with Static Analysis [**?**] tackles the same problem as this paper, but using static analysis to determine which load instructions to label as 'PND'. It demonstrates small IPC gains in select cases, but with an overall best-case gmean

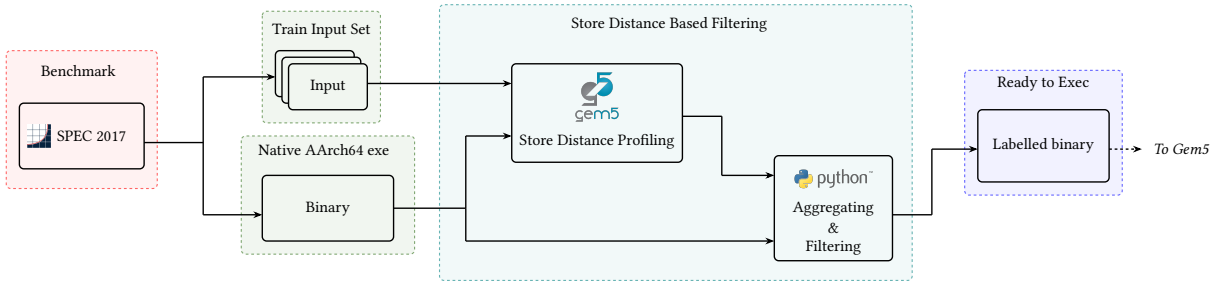

Figure 1: Toolflow for Store Distance based load labelling

of less than 0.1%. Furthermore, due to relying on static analysis it also sees performance regressions elsewhere, making it unviable. In comparison, by leveraging profiles we are able to achieve higher IPC gains in broader contexts. Our technique is also resistant to regressions on larger or more sophisticated MDP algorithms such as PHAST, which [**?**] is not.

Feedback-directed Memory Disambiguation Through Store Distance Analysis [**?**] proposes a similar profile-guided co-design to this paper, but with a more aggressive use case. In their work, the MDP is replaced altogether and load instructions are extended to include 4-bit distance fields indicating the store queue position they're likely to be dependent on. This achieves high accuracy compared to Store Sets, but comes at the cost of higher instruction bandwidth to encode predicted store distances, which is usually infeasible in modern processor designs where instruction bandwidth is critical. Larger processors would also likely require more bits to encode longer store queue distances. Furthermore, [**?**] could be more sensitive to profile accuracy than our work, as loads which do not appear in the profile must be assumed to be memory independent. In contrast, our technique is able to leave unseen loads to execute as normal.

Software-hardware Cooperative Memory Disambiguation [**?**] is a binary analysis co-design to reduce load queue pressure. Certain kinds of provably memory independent loads are labelled and prevented from being inserted into the load queue when issued. This technique is able to label 40% of static loads across Spec2006 floatspeed (but dynamic percentage is unclear). This also incurs additional instruction bandwidth by inserting barrier-like instructions when entering loops to ensure labelled loads are only removed from the load queue when their parent loop reaches a steady state in the pipeline. LSQ entries must also store an additional bit to track whether these custom instructions are in-flight or not. This work presents an interesting proposal for reducing load queue pressure in area-constrained processors that lack space for hardware-based load elimination techniques, but has reduced viability due to currently lacking a mechanism for ensuring memory coherence in multi-core systems.

Constable: Improving Performance and Power Efficiency by Safely Eliminating Load Instruction Execution [**?**] is a pure hardware technique to track and eliminate stable loads which consistently read the same value from memory. This technique is able to eliminate both the memory read and address calculation altogether, greatly improving pipeline efficiency. It also demonstrates a wide coverage a variety of workloads, improving gmean IPC by 5.1%. For larger performance-orientated processors, this technique would most likely be more effective at improving the efficiency of load execution than our work, with a large overlap (albeit not total) of the types of load instructions targeted. However, at an estimated hardware overhead of 12.4KB, this technique may be infeasible for the area-constrained cores we target.

Other Memory Dependence Prediction Algorithms Besides Store Sets and PHAST [**? ?**], other MDP algorithms include Store Vectors, NoSQ and MDP-TAGE [**? ? ?**]. Each of these algorithms offer different trade-offs and advantages. We choose to evaluate XS Store Sets and PHAST because we understand them to be the optimal choices for small and large cores respectively, and that XS Store Sets provides, to the best of our knowledge, the closest representation of MDP algorithms found in real processors.
NOTE: MDP-TAGE boasts the accuracy of a large store sets for only 70 bits of storage. I *think* that this is predicated on having a good branch predictor though, which the small cores we're targeting wouldn't have (although I haven't tested this). 1. does this make sense and 2. should we state it explicitly? If it does make sense, then its already contained implicitly in referring to XS Store Sets as 'optimal' for small cores (XS-SS also has the benefit of existing in a real CPU). If MDP-TAGE does work well for small cores though I think we kind of have to be evaluating that instead.

4 Method

This section presents the profile-guided co-design technique used in this paper. We put forward the concepts of store distances, distance thresholds, and how these are combined to find reliably memory independent loads. We then motivate the use of profiles to determine load labels, outlining the benefits provided over static analysis.

4.1 Store Distances & Labelling Thresholds

As overviewed in Section **??**, when loads are issued in out-of-order execution they search the store queue (SQ) for older stores with overlapping addresses. We use the term store distance to refer to the number of prior entries in the SQ a load's dependent store is found. A load that depends on the most recent prior store would have a store distance of 1.

The profile-derived store distance for each load is found by recording every per-execution store distance on train inputs. Store distances that occur in 95% or more executions are selected as that

load's profiled store distance. When no single store distance occurs in at least 95% of executions, the shortest observed distance is chosen.

The premise our technique is based on is that loads with either no or consistently long dependent store distances are good candidates to label as PND. This heuristic captures three different types of behaviour:

- **(1) Memory Independent Loads:** Loads which read constant data (i.e. have no store distance).
- **(2) Rarely Dependent Loads:** Loads which observe short dependent store distances in <5% of executions.
- **(3) Structurally Independent Loads:** Loads whose dependent stores do not exist in-flight at the same time the load is issued.
- **(4) Fast-to-Resolve Dependencies:** Loads which have in-flight but resolved dependent stores, allowing forwarding.

Loads with behaviour types one and two can always be safely labelled, but whether a load's store distance will fall into types three or four will depend on the processor being targeted. A larger processor with a longer SQ will hold more in-flight stores at once, so only loads with longer store distances will fall into this category. Type four is even further target (or even workload) specific, but is still correlated with a longer distance as more time is left between dispatching the store and issuing the load.

The optimal store distance threshold for a specific target processor is found via binary search, choosing the threshold with the best average performance over train inputs. By using a representative selection of workloads to perform the search with, an optimal store distance threshold can target the processor rather than the each individual workload, meaning the search only has to be performed once. Figure 2 shows how the performance impact of labels for different core sizes varies, and the optimal choice for each target.

Once a threshold is selected it is applied to each workloads profile, and loads with sufficiently long distances have their opcodes changed to the appropriate PND load variant.

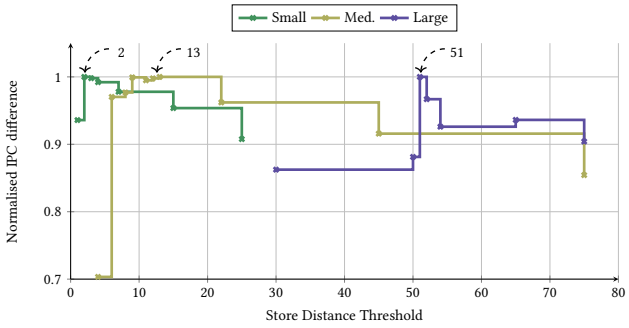

Figure 2: Store Distance threshold search for each CPU model. The Arrows indicate the best threshold.

4.2 Why Profiles?

While profile-guided optimisations (PGO) have long offered performance benefits, the technique has historically struggled to achieve adoption due to complicating the compilation workflow [**?**]. PGO also introduces concerns about the accuracy of generated profiles, and the potential to harm performance should real input differ too far from train input.

Addressing adoption, prior work shows that use of PGO has risen significantly in recent years [**?**]. This can be seen in enterprise projects such as Firefox and Chrome web browsers, the Python interpreter and the GCC compiler. Performance-critical server workloads have also seen renewed attention to the benefits that PGO can provide [**?**]. This enforces the use of profiles as a reasonable approach to designing new software-hardware co-designs.

We furthermore demonstrate in Section **??** that the our profiles generated from Spec2017 train inputs do generalise effectively to the test inputs, suggesting our proposed technique is able to generalise to unseen inputs. It is also important to note that in the case of load instructions which are unseen in the train inputs, these are never labelled and so execute as normal. This allows our technique to be more tolerant to workloads with high code coverage variation between inputs.

Lastly we highlight the benefits that profiles are able to provide to this technique over static analysis. Of the four types of load behaviour referred to in Section 4.1, only two of them (types one and three) could theoretically be captured by perfect alias analysis. Type two would then further require perfect memory dependence analysis, and there exists no analysis in conventional compiler design that attempts to capture the behaviour in type four. It is also important to note that the alias and dependence analysis offered by industrial compilers today is far from perfect [**?**]. Whereas stronger analyses do exist [**? ?**], these are either only applicable to domain-specific languages or do not scale to large programs, making them either less applicable or less feasible. In contrast, profiles allow much greater flexibility and coverage of load behaviour, which pairs well with speculative execution where mistakes will not invalidate program semantics.

5 Evaluation

This section presents experimental results using our profile-guided co-design. It highlights how performance is improved by eliminating false memory dependencies, the extent of redundant memory dependence prediction queries and associated power savings, as well as an estimation of the impact of reducing predictor read port pressure.

-todo: mention the other sections to be filled in later (profile analysis, prior work comparisons, sensitivity analysis etc)

5.1 Experimental Design

- todo: test phast on medium once store set size decided

We use Gem5 to simulate varying core size configurations to measure impact as components scale, going from a very small core (ROB=64) to a large core (ROB=512). Each core either utilises the XS Store Sets or PHAST predictor, depending on which provides better performance for the area budget of that size. We find that XS Store Sets performs optimally in smaller area allocations, whereas PHAST is more accurate when afforded larger sizes.

The full configuration for each model is detailed in Table 1. We evaluate the Spec2017 Intspeed and Floatspeed benchmarks using up to 10 simpoints, with an interval of 100M instructions and a

Table 1: Parameters of processor components for each simulated model

		V. Small	Small	Med.	Large
Instruction Window	ROB:	32	128	256	512
	IQ:	52	77	154	308
	LQ:	21	38	77	154
	SQ:	12	22	54	108
Pipeline Width	Fetch/	3	4	6	8
	Commit:	6	8	8	12
	Issue:				
L1 Cache	L1D (KB):	16	32	64	64
	L1I (KB):	16	32	32	64
	MSHRs:	4	8	16	32
L2 Cache	L2 (MB):	1	2	4	8
	MSHRs:	8	16	32	64
XS Store Sets	SSIT:	16	32	128	N/A
	LFST:	16	16	64	
	Slots:	1	2	2	
Clear Period:		125k			N/A
PHAST	Rows:	N/A	N/A	N/A	32
	Assoc:				2
	Tag Bits:				8
MDP Read Ports:		1	1	2	3
TAGE-SC-L Size:		8KB		64KB	
Prefetcher:		DCPT			

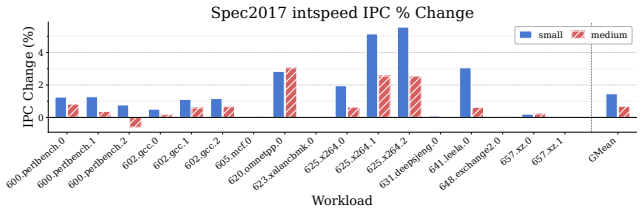

Figure 3: IPC % changes for each CPU model on each intspeed benchmark.

warm-up of 10M. Gem5 is modified to bypass MDP lookups for labelled loads and to avoid creating new entries in the case of a memory order violation. Violating labelled loads still roll back as normal and do not alter program semantics. The Gem5 results are subsequently processed by an extended McPat, which models XS Store Sets or PHAST as applicable.

5.2 Performance

Figure 3 shows the percent changes in IPC across Spec2017 intspeed for each core configuration. The gmean IPC gain ranges from 1.4%, 0.7% and 0% as core size grows. This demonstrates our techniques ability to deliver zero-cost performance gains on area-constrained cores while still maintaining performance on larger cores with more sophisticated MDPs. Examining the gains in the best case (small core size), the IPC gains have an uneven spread across benchmarks, with some seeing large benefits and others next to none. This loosely correlates to the magnitude of false dependencies before and after

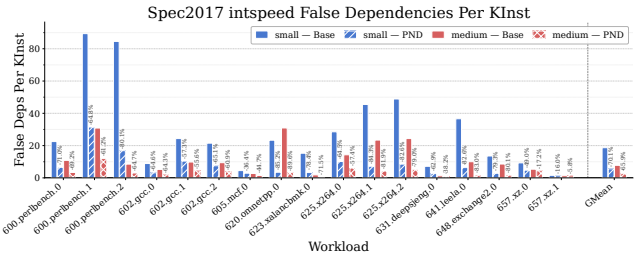

Figure 4: Percent change of false dependencies.

seen in Figure 4. The four largest gainers - 625.*x*264_*s* inputs 1 and 2, 641.*leela_s* and 620.*omnetpp_s* - all exhibit relatively higher false dependencies per kilo-instruction in the base case, and see this cut down significantly by over 80% when PND labels are used. As the core size increases to the medium configuration, most of these gains are diminished as the larger Store Set tables reduce the base number of hash collisions. A sensitivity analysis on the size of Store Sets on the small configuration results is shown in Section 5.6.

620.*omnetpp_s* exhibits more false dependencies per kilo-instruction in the medium core despite the larger Store Set size. This is likely because while it doesn't issue the most loads overall, it does have the highest number of dependent loads. Whereas the average across intspeed benchmarks is approximately 2 million dependent loads, 620.*omnetpp_s* observes 6 million. This suggests due to the nature of the workload, a longer instruction window captures more dependencies which the larger Store Set tables cannot offset.

In select cases PND labels can lead to a performance regression, notably for input 2 of 600.*perlbench_s*. This is because loads have been mis-labelled from the train inputs and cause an increase in memory order violations on the test inputs, as seen in Figure 5 for the medium core configuration. Although memory order violations are increased in all workloads, they're mostly offset by the reduction in false dependencies. It is also worth noting that violations are already very rare events, measured in mega-instructions rather than kilo-instructions, so even a 100% increase can often still keep them in negligible ranges compared to the number of false dependencies. In input 2 of 600.*perlbench_s* though the absolute number of violations is at it's highest, and so offsets any reductions in false dependencies (which as was also already discussed have more a limited impact in this workload).

The degree to which violations increase is controlled by the threshold value discussed in Section **??**. The optimal threshold may still provide a net performance gain while still causing small regressions in specific workloads, and a larger threshold could instead be chosen to ensure regressions never occur. They could be also mitigated by specialising the PGO technique to each benchmark rather than the processor.

5.3 Power

- analyse power savings, correlated to lookup reduction. small core has small shift in MDP power, but total core power saving congruent to IPC gain, which is an opportuny to remind how the gains come at no area/power tradeoff.

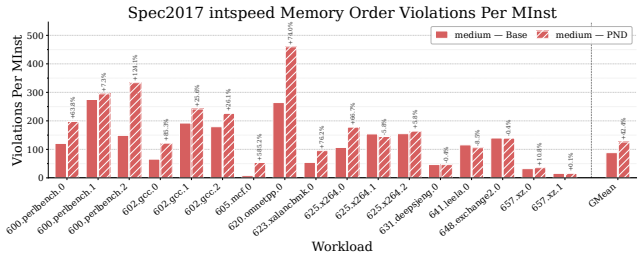

Figure 5: Percent change of memory order violations. Values roughly equal between core sizes.

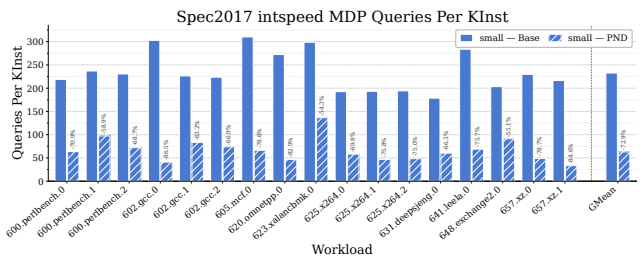

Figure 6: Percent change of MDP queries. Values roughly equal between core sizes.

- need large core results which have a larger MDP power saving, but next to no core change. can try and speculate how this offsets growing MDP power for the future
- need large core with big phast power savings which will hopefully nudge total core power a little more? not sure where to go with this section if it doesn't tho.

5.4 Read Port Impact Estimation

Real processors have a limited number of read ports for each predictor component. If the memory dependence predictor has two ports, the processor can dispatch up to two memory operations per cycle. Gem5 does not model this behaviour by default, as it is a low-level detail that is difficult to represent accurately at the level of abstraction Gem5 simulates. Given the importance of this aspect to our proposed technique, we provide an estimation of the potential impact on IPC when read ports are limited. Each core configuration is assigned a number of read ports to the memory dependence predictor as listed in Table 1. The dispatch stage is modelled to block when the number of memory operations dispatched within a cycle exceeds the maximum number of read ports. This includes both loads and stores for XS Store Sets and only loads for PHAST. It is assumed that predictions always arrive within a single cycle for both XS Store Sets and PHAST, and so read port usage is reset to zero every cycle. This provides a rough estimation of the impact of predicted non-dependent loads on workloads with a high density of memory operations. For clarity, the number of MDP read ports does **not affect** any other results presented in this paper.

- insert results here

5.5 Profile Analysis & Threshold Sensitivity

- need to generate profile from test input for this, will take a few days

5.6 Store Set Table Size Sensitivity

As seen in Section 5.2, the number of false dependencies falls sharply when increasing the Store Set table sizes along with core size. We present a sensitivity study for how gmean IPC improvement and base false dependencies in the small core configuration varies as both SSIT and LFST sizes grow.

- graphs here

Figure ?? shows that the majority of false dependencies can be removed by growing the Store Set tables, and likewise Figure ?? shows that the IPC gain diminishes along with this too. This demonstrates the main source of performance our technique offers is due to reducing hash collisions in the predictor tables, but without having to grow them.

Furthermore, we perform an iso-area comparison in the small core configuration between doubling the Store Set size verses growing the load queue by an equal number of bits. Whereas growing Store Sets provide an additional x% gmean IPC, the larger load queue provides y%. This suggests that Store Set hash collisions would not typically be eliminated in small processors by growing the predictor size.

6 Threats to Validity

- encoding space (cite cooperative memory disambig)
- cpu models (and gem5)
- intended for specific target compilation

7 Future Work

-per benchmark tuning & JITs
-serverless workloads

8 Conclusion